

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра програмної інженерії

ЗВІТ
з лабораторної роботи №4
з дисципліни «Основи програмної інженерії»
на тему «Рефакторинг коду та проведення Code Review
за індустріальними стандартами»

Виконали:

студенти групи ПЗПІ-25-4
команда-4-8

Платіцина К.

Михайліченко З.

Тараторін Т.

Моргун Н.

Перевірив:

стр. викл. каф. ШІ

Гребенюк В.О.

Харків 2026

Тема: Рефакторинг коду та проведення Code Review за індустріальними стандартами.

Мета: Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

1. Результати Code Review одногрупника

Під час перехресного рев'ю коду, яке виконав член команди Тимофій Тараторін, були виявлені наступні недоліки:

Таблиця 1 – Результати проведеного Code Review

№	Рядок коду	Проблема	Категорія	Рекомендація
1	10-11	Жорстко закодовані значення 1400 та 2030	Magic Numbers	Винести числа у константи класу (наприклад, MIN_YEAR та MAX_YEAR).
2	16-17	Дві різні книги з однаковою назвою від різних авторів вважаються дублікатами	Logical Flaw / Incomplete implementation	Додати перевірку за автором: <code>c.author === author</code> .
3	33	Змінна <code>c</code> занадто коротка і не описує контекст	Meaningless Variable Names	Перейменувати <code>c</code> на <code>citation</code> або <code>item</code> для кращої читабельності.
4	36	Ланцюжок <code>if-else if</code> для вибору формату стилю погано масштабується	Complex Conditional	Використати об'єкт-словник (Strategy pattern) або конструкцію <code>switch-case</code> .
5	4	Рядки стилів дублюються в масиві та в методі генерації	Primitive Obsession / Duplicated Code	Винести стилі у єдиний об'єкт, щоб уникнути помилок у майбутньому.

2. Посилання на PR з коментарями Code Review (URL)

<https://github.com/platitsynakseniia21/YesCite---Group-4-8/pull/2>

3. Результати рефакторингу власного коду

Рефакторинг коду, усунення вразливостей та впровадження нових архітектурних рішень на основі результатів Code Review були виконані членом команди Платіциною Ксенією.

Операція 1: Усунення магічних чисел

БУЛО: `if (year < 1400 || year > 2030) { throw new Error(...); }`

СТАЛО: `static MIN_YEAR = 1400; static MAX_YEAR = 2030; if (year < CitationManager.MIN_YEAR || year > CitationManager.MAX_YEAR) { ... }`

ЧОМУ: Позбулися жорстко закодованих значень. Тепер межі валідації років знаходяться в одному місці, що робить код стійким до змін бізнес-логіки та полегшує його підтримку.

Операція 2: Впровадження патерну Strategy

БУЛО: Використання масиву `this.validStyles` та багаторівневих перевірок `if-else` всередині циклу для форматування бібліографії.

СТАЛО: Створено об'єкт-словник: `const BIBLIOGRAPHY_FORMATTERS = {`
`APA: (citation) => ..., MLA: (citation) => ... }`; Застосування:
`.map(citation => BIBLIOGRAPHY_FORMATTERS[style](citation));`

ЧОМУ: Вирішено проблему масштабування (дотримання Open/Closed Principle). Додавання нових стилів цитування тепер не вимагає модифікації логіки самого методу `generateBibliography`.

Операція 3: Виправлення логічної помилки та неймінгу

БУЛО: `const isDuplicate = this.citations.some(c => c.title === title && c.year === year);`

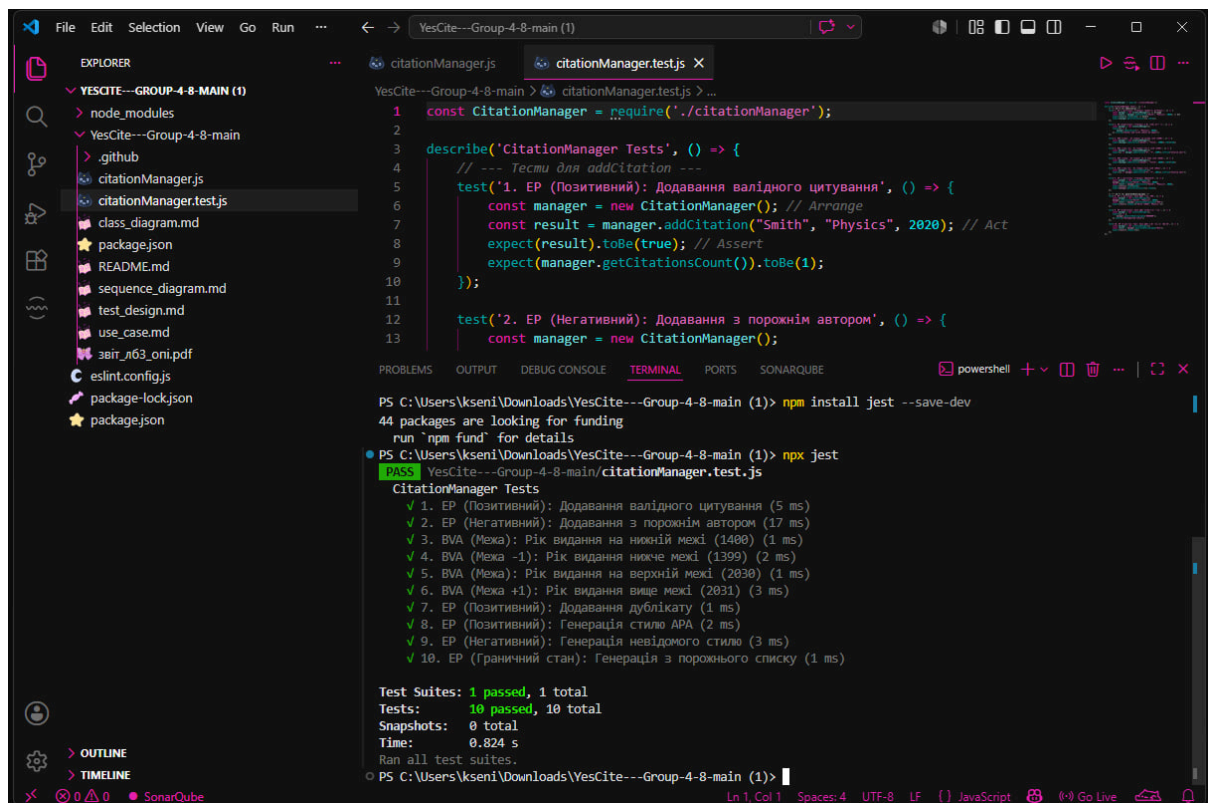
СТАЛО: `const isDuplicate = this.citations.some(citation => citation.title === title && citation.year === year && citation.author === author);`

ЧОМУ: Виправлено баг, через який втрачалися унікальні книги з однаковими назвами від різних авторів. Замість неінформативної змінної `s` використано `citation`, що підвищує зрозумілість коду.

4. Звіт регресійного тестування

Налаштування середовища та запуск регресійних тестів після рефакторингу були виконані членом команди Моргун Надією.

Після внесення архітектурних змін (впровадження патерну Strategy та усунення магічних чисел), було проведено регресійне тестування за допомогою фреймворку Jest. Усі 10 із 10 тестів успішно пройдені. Це підтверджує, що оптимізація коду не порушила початкову бізнес-логіку програми, а всі виявлені під час Code Review помилки були усунуті без створення нових багів.



```
1 const CitationManager = require('./citationManager');
2
3 describe('CitationManager Tests', () => {
4   // --- Тести для addCitation ---
5   test('1. EP (Позитивний): Додавання валідного цитування', () => {
6     const manager = new CitationManager(); // Arrange
7     const result = manager.addCitation("Smith", "Physics", 2020); // Act
8     expect(result).toBe(true); // Assert
9     expect(manager.getCitationsCount()).toBe(1);
10  });
11
12  test('2. EP (Негативний): Додавання з порожнім автором', () => {
13    const manager = new CitationManager();
```

```
PS C:\Users\kseni\Downloads\YesCite---Group-4-8-main (1)> npm install jest --save-dev
44 packages are looking for funding
  run `npm fund` for details
PS C:\Users\kseni\Downloads\YesCite---Group-4-8-main (1)> npx jest
PASS YesCite---Group-4-8-main/citationManager.test.js
CitationManager Tests
  ✓ 1. EP (Позитивний): Додавання валідного цитування (5 ms)
  ✓ 2. EP (Негативний): Додавання з порожнім автором (17 ms)
  ✓ 3. BVA (Межа): Рік видання на нижній межі (1400) (1 ms)
  ✓ 4. BVA (Межа -1): Рік видання нижче межі (1399) (2 ms)
  ✓ 5. BVA (Межа): Рік видання на верхній межі (2030) (1 ms)
  ✓ 6. BVA (Межа +1): Рік видання вище межі (2031) (3 ms)
  ✓ 7. EP (Позитивний): Додавання дублікату (1 ms)
  ✓ 8. EP (Позитивний): Генерація стилю APA (2 ms)
  ✓ 9. EP (Негативний): Генерація невідомого стилю (3 ms)
  ✓ 10. EP (Граничний стан): Генерація з порожнього списку (1 ms)

Test Suites: 1 passed, 1 total
Tests: 10 passed, 10 total
Snapshots: 0 total
Time: 0.024 s
Ran all test suites.
```

Рис.1 — Результати регресійного тестування у фреймворку Jest

5. Порівняння метрик «ДО / ПІСЛЯ»

Збір та аналіз метрик якості коду до рефакторингу виконали Моргун Надія (налаштування візуального та консольного лінтерів SonarLint і ESLint) та Михайліченко Злата (аналіз цикломатичної складності за допомогою утиліти Lizard). Фінальний збір метрик оптимізованого коду проведено Платіциною Ксенією.

Візуальні докази стану коду "ДО" рефакторингу:

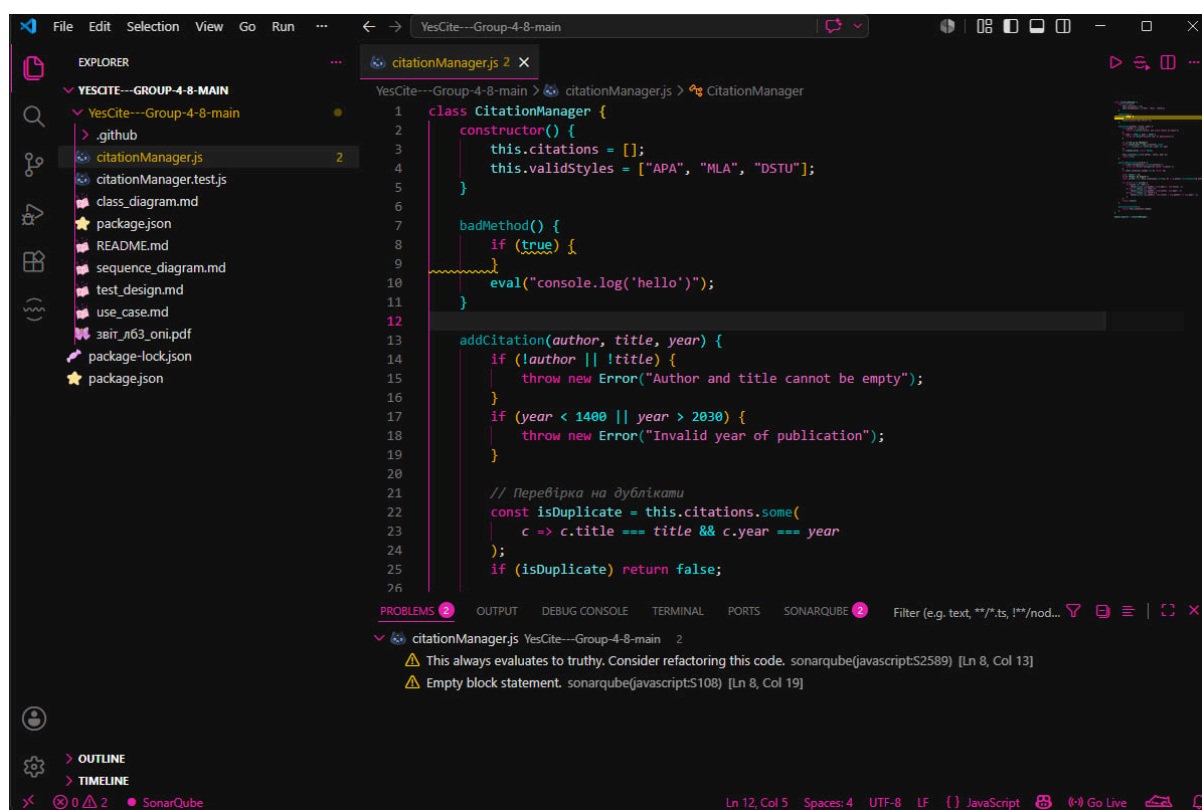


Рис.2 — Помилки архітектури та синтаксису, знайдені плагіном SonarQube

```

PS C:\Users\kseni\Downloads\YesCite---Group-4-8-main> npx eslint **/citationManager.js
(node:15164) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file:///C:/Users/kseni/Downloads/YesCite---Group-4-8-main/eslint.config.js?mtime=1780481037
878 is not specified and it doesn't parse as CommonJS.
Reparsing as ES module because module syntax was detected. This incurs a performance overhead.
To eliminate this warning, add "type": "module" to C:\Users\kseni\Downloads\YesCite---Group-4-8-main\package.json.
(Use `node --trace-warnings ...` to show where the warning was created)

C:\Users\kseni\Downloads\YesCite---Group-4-8-main\YesCite---Group-4-8-main\citationManager.js
   8:19  error  Empty block statement  no-empty
  10:9   error  `eval` can be harmful  no-eval

✖ 2 problems (2 errors, 0 warnings)

PS C:\Users\kseni\Downloads\YesCite---Group-4-8-main>

```

Рис.3 — Помилки no-empty та no-eval, зафіксовані консольним лінтером ESLint

```

PS C:\Users\zmiha\Downloads\YesCite---Group-4-8-main\YesCite---Grou
p-4-8-main> py -m lizard citationManager.js
=====
  NLOC    CCN    token  PARAM  length  location
-----
      4      1     23      0       4  constructor@2-5@citationManag
er.js
      2      2     12      0       2  (anonymous)@17-18@citationMan
ager.js
     13      6     83      3      17  addCitation@7-23@citationMana
ger.js
      1      1     11      0       1  (anonymous)@33-33@citationMan
ager.js
     18      7    173      1      21  generateBibliography@25-45@ci
tationManager.js
      3      1     11      0       3  getCitationsCount@47-49@citat
ionManager.js
1 file analyzed.
=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     42      6.8    3.0     52.2         6  citationManager.
js
=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000
or nloc > 1000000 or parameter_count > 100)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  F
un Rt   nloc Rt
-----
     42      6.8    3.0     52.2         6         0
0.00    0.00

```

Рис.4 — Висока цикломатична складність (CCN = 7) методу generateBibliography до рефакторингу

Таблиця 2 — Порівняння метрик якості коду

Метрика	Значення «ДО»	Значення «ПІСЛЯ»	Динаміка / Коментар
Кількість SonarLint issues	2 помилки	0 помилок	Усунуто проблеми S108 (Empty block) та S2589. Додатково вирішено S7727 під час оптимізації методу .map().
Кількість ESLint warnings/errors	2 помилки	0 помилок	Усунуто використання заборонених конструкцій no-eval та no-empty.
Цикломатична складність (CCN)	CCN (generateBibliography) = 7 Середня складність = 3.0	CCN (generateBibliography) = 3 Середня складність = 1.9	Завдяки впровадженню об'єкта-словника (патерн Strategy) різко зменшилась заплутаність коду та кількість розгалужень.

Візуальні докази стану коду "ПІСЛЯ" рефакторингу:

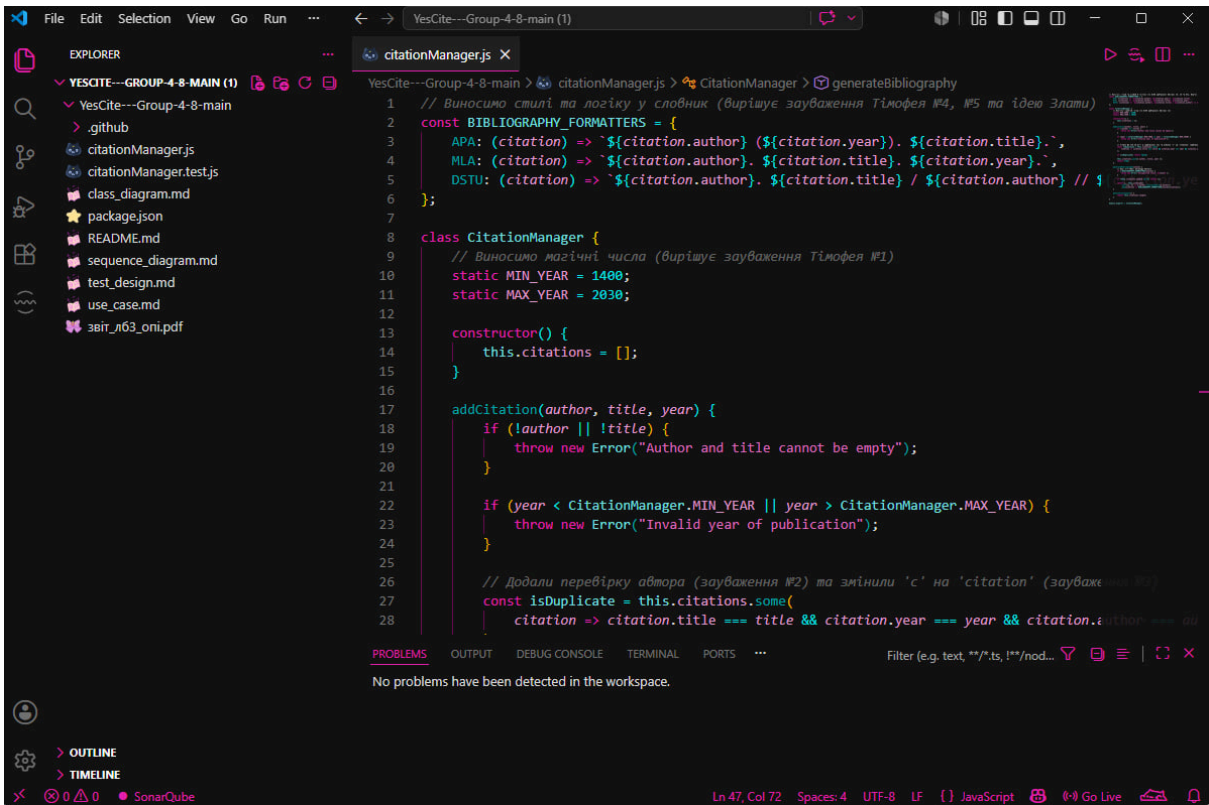


Рис.5 — Відсутність архітектурних помилок у SonarQube після оптимізації

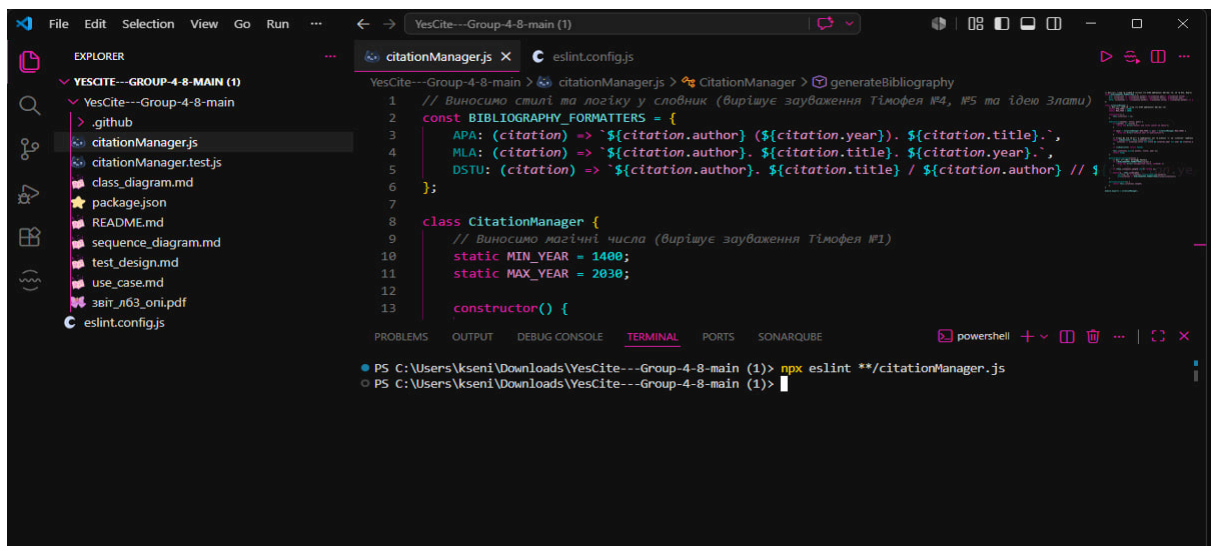


Рис.6 — Успішне проходження перевірки консольним лінтером ESLint
(0 помилок)

```
PS C:\Users\zmiha\Downloads\YesCite---Group-4-8-main\YesCite---Group-4-8-main> py -m lizard citationManager.js
=====
NLOC   CCN   token  PARAM  length  location
-----
      2     1    20     1      2  APA@3-4@citationManager.js
      2     1    20     1      2  MLA@4-5@citationManager.js
      2     1    23     1      2  DSTU@5-6@citationManager.js
      3     1    11     0      3  constructor@13-15@citationManager
.js
      2     3    18     0      2  (anonymous)@28-29@citationManager
.js
     13     6    87     3     19  addCitation@17-35@citationManager
.js
      1     1    11     0      1  (anonymous)@46-46@citationManager
.js
      1     1     8     0      1  (anonymous)@47-47@citationManager
.js
      9     3    65     1     12  generateBibliography@37-48@citati
onManager.js
      3     1    11     0      3  getCitationsCount@50-52@citationM
anager.js
1 file analyzed.
=====
NLOC   Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     39      3.8    1.9    27.4         10  citationManager.js
=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or
nloc > 1000000 or parameter_count > 100)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun R
t   nloc Rt
-----
     39      3.8    1.9    27.4         10         0      0.00
0.00
```

Рисунок 7 — Зниження цикломатичної складності (CCN = 3) після
застосування патерну Strategy

6. Підсумкова рефлексія

Процес Code Review виявився критично важливим етапом розробки, оскільки "замилений" погляд автора часто ігнорує архітектурні недоліки. Найціннішим досвідом стало усвідомлення того, що код може ідеально працювати, але при цьому бути погано масштабованим (наприклад, ланцюжки if-else). Спільне використання ручного рев'ю (для пошуку логічних багів) та автоматизованих інструментів (SonarLint, ESLint, Lizard) дозволило не лише знайти вразливості, але й довести якість рефакторингу через об'єктивне зниження цикломатичної складності.

Розподіл завдань у команді:

Тараторін Т.: Проведення перехресного Code Review коду одногрупників, виявлення логічних та архітектурних недоліків, оформлення структурованих зауважень у Pull Request на GitHub (пункти 2 та 3 звіту).

Платіцина К.: Проведення рефакторингу проблемного коду (усунення "магічних чисел", впровадження архітектурного патерну Strategy, виправлення логічних помилок пошуку дублікатів), налаштування та успішний запуск регресійного тестування у фреймворку Jest, збір фінальних метрик "чистого" коду (пункти 4, 5 та частина пункту 6 звіту).

Моргун Н.: Налаштування та аналіз вихідного коду за допомогою статичного аналізатора SonarQube та консольного літера ESLint, фіксація метрик коду до проведення оптимізації (частина пункту 6 звіту).

Михайліченко З.: Вимірювання цикломатичної складності коду за допомогою утиліти Lizard до та після рефакторингу (частина пункту 6).